



PRACTICAL ASSIGNMENT-1



Name :- Prasad Manisha Vijaykumar

Subject :- Dynamic web development using react

Subject Code :- 23PGCA104

Practical Assignment :- 1

Class :- MCA

Division :- Y1

Roll No :- 42

Date :- 14-09-2026

1. ES6 Integrated Programming Problem

Develop an ES6 JavaScript program to manage student information.

The program should:

- Create a Student class with name, rollNo, and marks.
- Use a constructor to initialize the student details.
- Create an array containing at least five student objects.
- Use filter() to identify students scoring more than 60.
- Use map() to generate an array containing only student names.
- Use reduce() to calculate the average marks.
- Use template literals to display the final results.

Code:

```
<html>
<body>
<script>
class Student
{
  constructor(name, rollNo, marks)
  {
    this.name = name;
    this.rollNo = rollNo;
    this.marks = marks;
  }
}

// Array of five student objects
const students = [
  new Student("Amit", 1, 75),
  new Student("Priya", 2, 55),
  new Student("Rahul", 3, 82),
  new Student("Neha", 4, 68),
  new Student("Kiran", 5, 45)
];

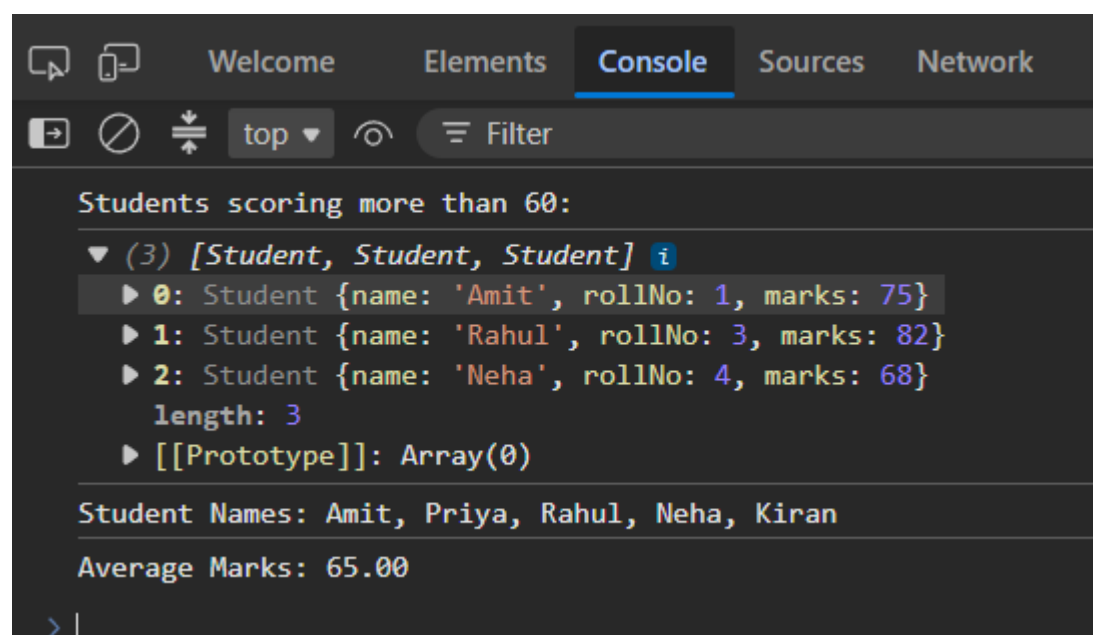
// Students scoring more than 60
const passedStudents = students.filter(student => student.marks > 60);

// Get only student names
const studentNames = students.map(student => student.name);
```

```
// Calculate average marks
const totalMarks = students.reduce((total, student) => total + student.marks,
0);
const averageMarks = totalMarks / students.length;

// Display results
console.log(`Students scoring more than 60:`);
console.log(passedStudents);
console.log(`Student Names: ${studentNames.join(", ")}`);
console.log(`Average Marks: ${averageMarks.toFixed(2)}`);
</script>
</body>
</html>
```

Output:



2. Promise and Async/Await Problem

Assume that a function `fetchStudentData()` returns student information after a delay of 2 seconds.

Write an ES6 program that:

- Creates a Promise to simulate data retrieval.
- Resolves the Promise when data is successfully retrieved.
- Rejects the Promise when an error occurs.
- Handles the Promise using `async` and `await`.
- Uses `try...catch` to handle errors.
- Displays an appropriate message using template literals

Code:

```
<html>
<body>
<script>
function fetchStudentData()
{
    return new Promise((resolve, reject) => {

        setTimeout(() => {

            const success = true;

            if (success)
            {
                resolve({
                    name: "Amit",
                    rollNo: 101,
                    marks: 85
                });
            }
            else
            {
                reject("Failed to fetch student data");
            }

        }, 2000);
    });
}
```

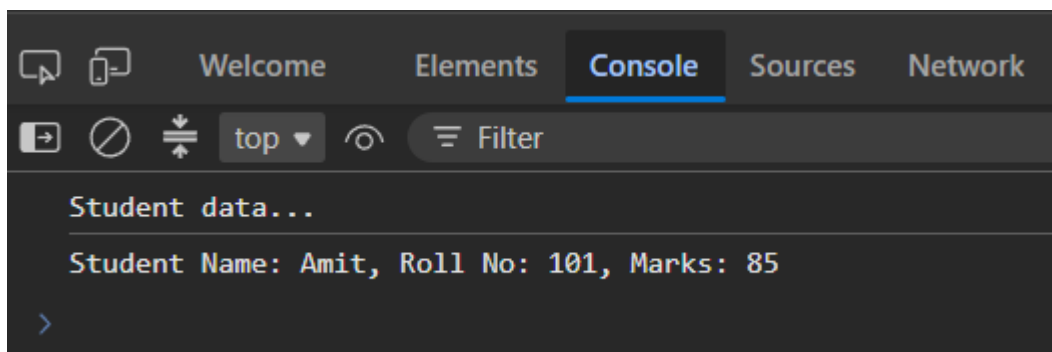
```
async function getStudentData()
{

    try
    {
        console.log(`Student data...`);
        const student = await fetchStudentData();

        console.log(
            `Student Name: ${student.name},
            Roll No: ${student.rollNo}, Marks: ${student.marks}`
        );
    }
    catch (error)
    {
        console.log(`Error: ${error}`);
    }
}

getStudentData();
</script>
</body>
</html>
```

Output:



3. Array Processing Challenge

Given the following array:

```
const products = [  
    {name: "Laptop", price: 50000},  
    {name: "Mouse", price: 800},  
    {name: "Keyboard", price: 1500},  
    {name: "Monitor", price: 12000},  
    {name: "Printer", price: 9000}  
];
```

Write an ES6 program to:

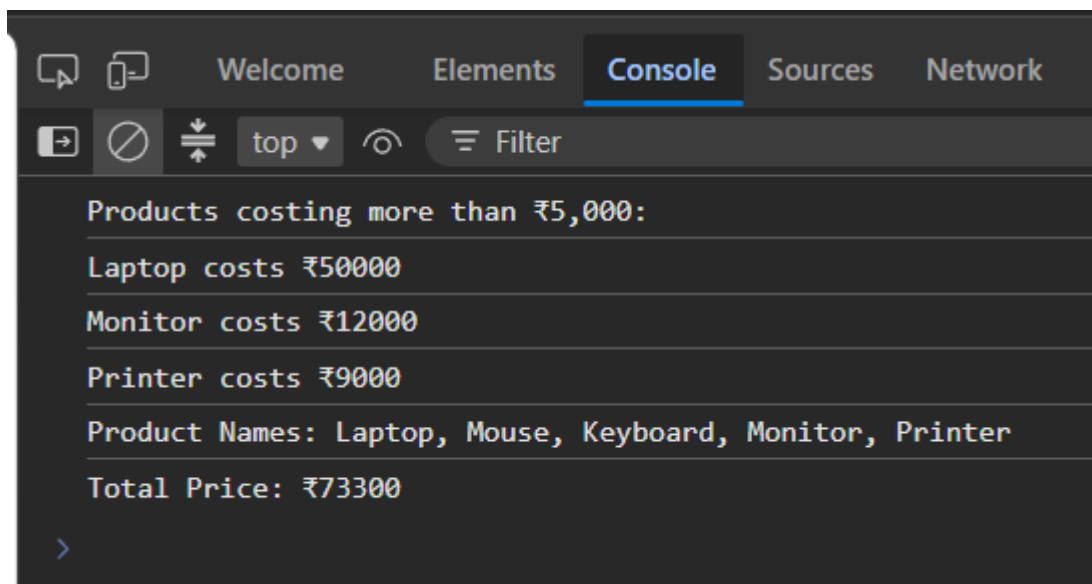
- Use `filter()` to identify products costing more than ₹5,000.
- Use `map()` to extract product names.
- Use `reduce()` to calculate the total price.
- Use destructuring to extract name and price.
- Use template literals to display the result.

Code:

```
<html>  
<body>  
<script>  
const products = [  
    { name: "Laptop", price: 50000 },  
    { name: "Mouse", price: 800 },  
    { name: "Keyboard", price: 1500 },  
    { name: "Monitor", price: 12000 },  
    { name: "Printer", price: 9000 }  
];  
// Products costing more than ₹5,000  
const expensiveProducts = products.filter(product => product.price > 5000);  
// Extract product names  
const productNames = products.map(product => product.name);  
// Calculate total price  
const totalPrice = products.reduce((total, product) => total + product.price,  
0);
```

```
// Destructuring
console.log("Products costing more than ₹5,000:");
for (const product of expensiveProducts)
{
    const { name, price } = product;
    console.log(`${name} costs ₹${price}`);
}
console.log(`Product Names: ${productNames.join(", ")}`);
console.log(`Total Price: ₹${totalPrice}`);
</script>
</body>
</html>
```

Output:



4. Spread, Rest and Default Parameter Challenge

Write an ES6 program that defines a function `calculateTotal()` which accepts any number of product prices.

The function should:

- Use the rest operator to accept multiple prices.
- Use `reduce()` to calculate the total.
- Apply a default discount of 10% if no discount is supplied.
- Use the spread operator to combine two arrays of prices.
- Display the final amount using a template literal.

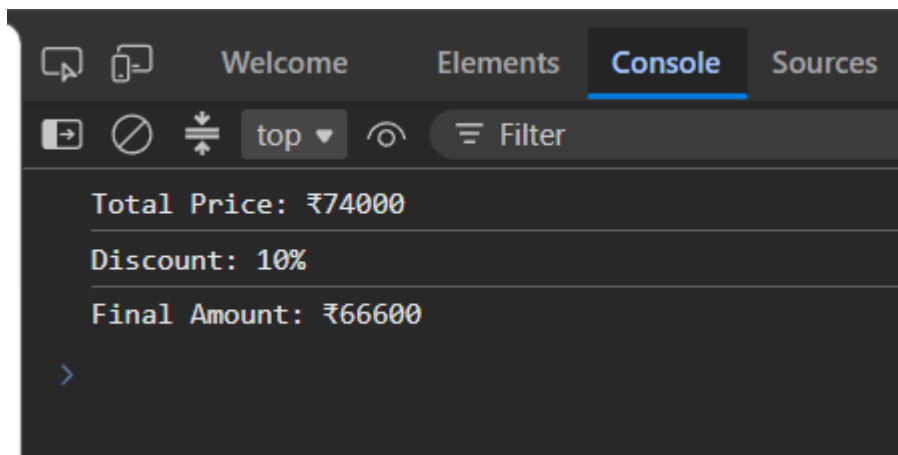
Code:

```
<html>
<body>
<script>
function calculateTotal(discount = 10, ...prices)
{
  const total = prices.reduce((sum, price) => sum + price, 0);
  const discountAmount = total * discount / 100;
  const finalAmount = total - discountAmount;

  console.log(`Total Price: ₹${total}`);
  console.log(`Discount: ${discount}%`);
  console.log(`Final Amount: ₹${finalAmount}`);
}

// Two arrays of product prices
const prices1 = [12000, 12000, 20000];
const prices2 = [20000, 10000];
// Spread operator combines the arrays
const allPrices = [...prices1, ...prices2];
// Rest operator receives all prices
calculateTotal(undefined, ...allPrices);
</script>
</body>
</html>
```

Output:



5. Module-Based Application

Design a simple ES6 module-based application for calculating student results.

Your application should contain:

`student.js`

- A function to calculate total marks.
- A function to calculate percentage.
- Export both functions.

`app.js`

- Import both functions.
- Create an array of marks.
- Calculate total and percentage.
- Display the result using template literals.

Code:

Prg5.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Program 5 Output</title>
</head>
<body>

<h2>Student Result</h2>

<script type="module" src="app.js"></script>

</body>
</html>
```

app.js

```
import { calculateTotal, calculatePercentage } from "./student.js";

const marks = [85, 70, 90, 80, 75];

const total = calculateTotal(marks);
const percentage = calculatePercentage(marks);

console.log(`Total Marks: ${total}`);
console.log(`Percentage: ${percentage.toFixed(2)}%`);
```

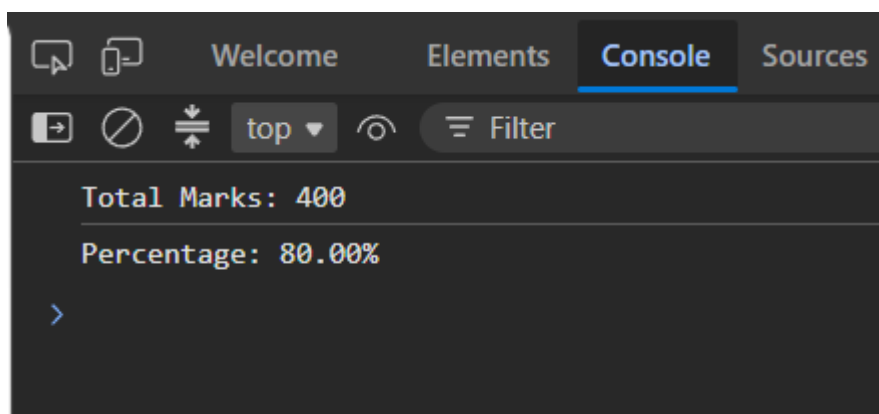
student.js

```
// Calculate total marks
export function calculateTotal(marks) {
  return marks.reduce((total, mark) => total + mark, 0);
}

// Calculate percentage
export function calculatePercentage(marks)
{
  const total = calculateTotal(marks);
  const maximumMarks = marks.length * 100;

  return (total / maximumMarks) * 100;
}
```

Output:



6. Map and Set Application

Develop an ES6 program for managing student attendance.

The program should:

- Use a Set to store unique student roll numbers.
- Demonstrate addition and deletion of students.
- Use a Map to associate roll numbers with student names.
- Retrieve a student's name using the roll number.
- Iterate through the Map using for-of.
- Display the total number of unique students.

Code:

```
<html>
<body>
<script>
// Set stores unique roll numbers
const rollNumbers = new Set();

rollNumbers.add(101);
rollNumbers.add(102);
rollNumbers.add(103);
rollNumbers.add(101); // Duplicate, will not be added

console.log(`Roll Numbers: ${[...rollNumbers]}`);

// Delete a student
rollNumbers.delete(102);

console.log(`After deleting roll number 102: ${[...rollNumbers]}`);

// Map associates roll number with student name
const students = new Map();

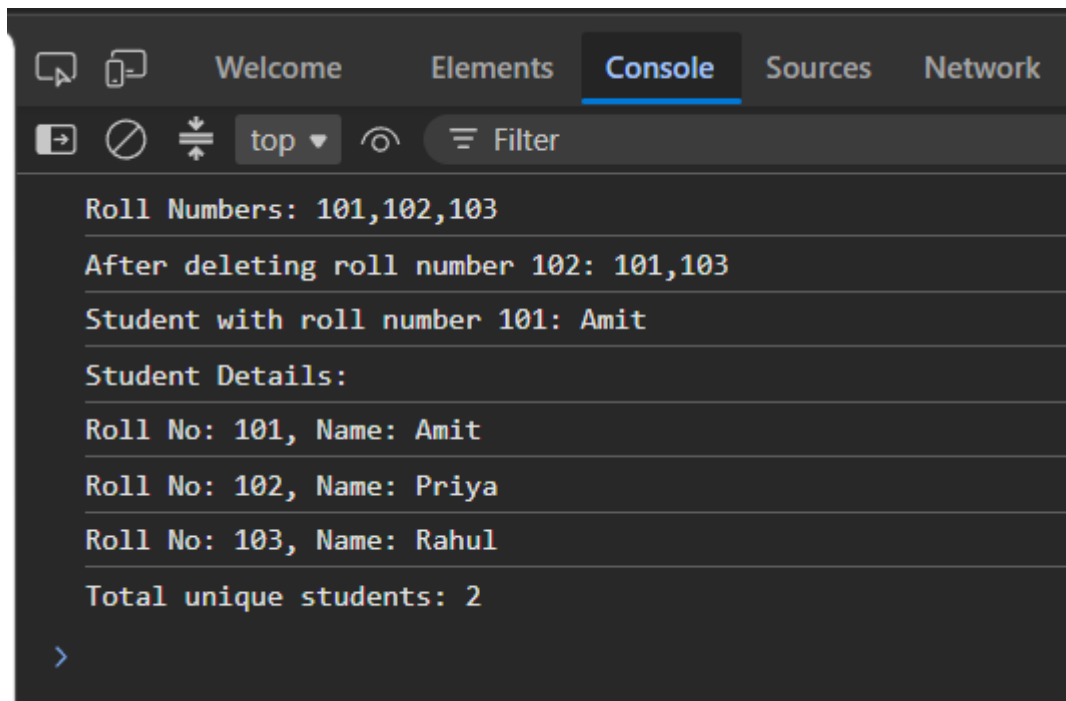
students.set(101, "Amit");
students.set(102, "Priya");
students.set(103, "Rahul");

// Retrieve student's name
console.log(`Student with roll number 101: ${students.get(101)}`);
console.log(`Student Details:`);
```

```
// Iterate through Map
for (const [rollNo, name] of students)
{
    console.log(`Roll No: ${rollNo}, Name: ${name}`);
}

// Total unique students
console.log(`Total unique students: ${rollNumbers.size}`);
</script>
</body>
</html>
```

Output:



7. this Keyword and Arrow Function Analysis

Consider the following code:

```
const student = {  
  name: "Amit",  
  regularFunction: function() {  
    console.log(this.name);  
  },  
  arrowFunction: () => {  
    console.log(this.name); }  
};  
student.regularFunction();  
student.arrowFunction();
```

Answer the following:

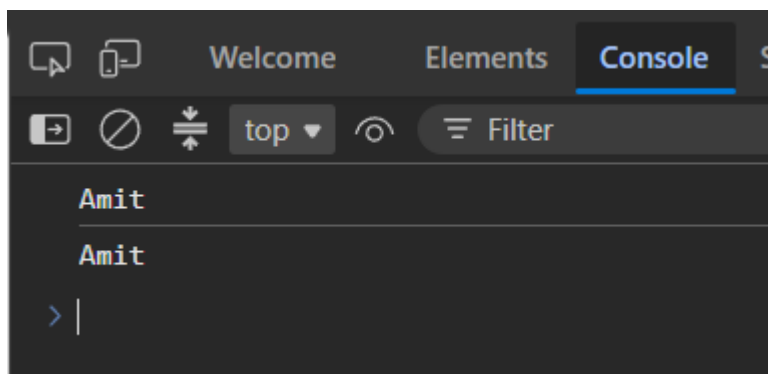
- Explain the behavior of this in regularFunction.
- Explain the behavior of this in arrowFunction.
- Explain how arrow functions differ from regular functions regarding this.
- Rewrite the code so that both functions correctly display the student's name.
- Explain your solution.

Code:

```
<html>
<body>
<script>
const student =
{
  name: "Amit",
  regularFunction: function()
  {
    console.log(this.name);
  },
  arrowFunction: function()
  {
    console.log(this.name);
  }
};
student.regularFunction();
student.arrowFunction();

</script>
</body>
</html>
```

Output:



8. Comprehensive ES6 Data Processing Challenge

Consider:

```
const students = [  
  {name: "A", marks: 85, course: "BCA"},  
  {name: "B", marks: 45, course: "BCA"},  
  {name: "C", marks: 72, course: "BSc"},  
  {name: "D", marks: 91, course: "BSc"},  
  {name: "E", marks: 38, course: "BCA"}  
];
```

Write an ES6 program to:

1. Extract students scoring more than 50 using filter().
2. Extract only student names using map().
3. Calculate the total marks using reduce().
4. Use destructuring to access student properties.
5. Use template literals to display each selected student's information.
6. Use a Set to obtain the unique courses.
6. Iterate through the students using a for-of loop.

Code:

```
<html>  
<body>  
<script>  
const students = [  
  { name: "A", marks: 85, course: "BCA" },  
  { name: "B", marks: 45, course: "BCA" },  
  { name: "C", marks: 72, course: "BSc" },  
  { name: "D", marks: 91, course: "BSc" },  
  { name: "E", marks: 38, course: "BCA" }  
];  
  
// 1. Students scoring more than 50  
const selectedStudents = students.filter(  
  (  
    student => student.marks > 50  
  )  
);
```

```
// 2. Extract student names
const studentNames = selectedStudents.map
(
  student => student.name
);

// 3. Calculate total marks
const totalMarks = students.reduce
(
  (total, student) => total + student.marks,
  0
);

// 4 & 5. Destructuring and template literals
console.log("Students scoring more than 50:");

for (const { name, marks, course } of selectedStudents)
{
  console.log(
    `Name: ${name}, Marks: ${marks}, Course: ${course}`
  );
}

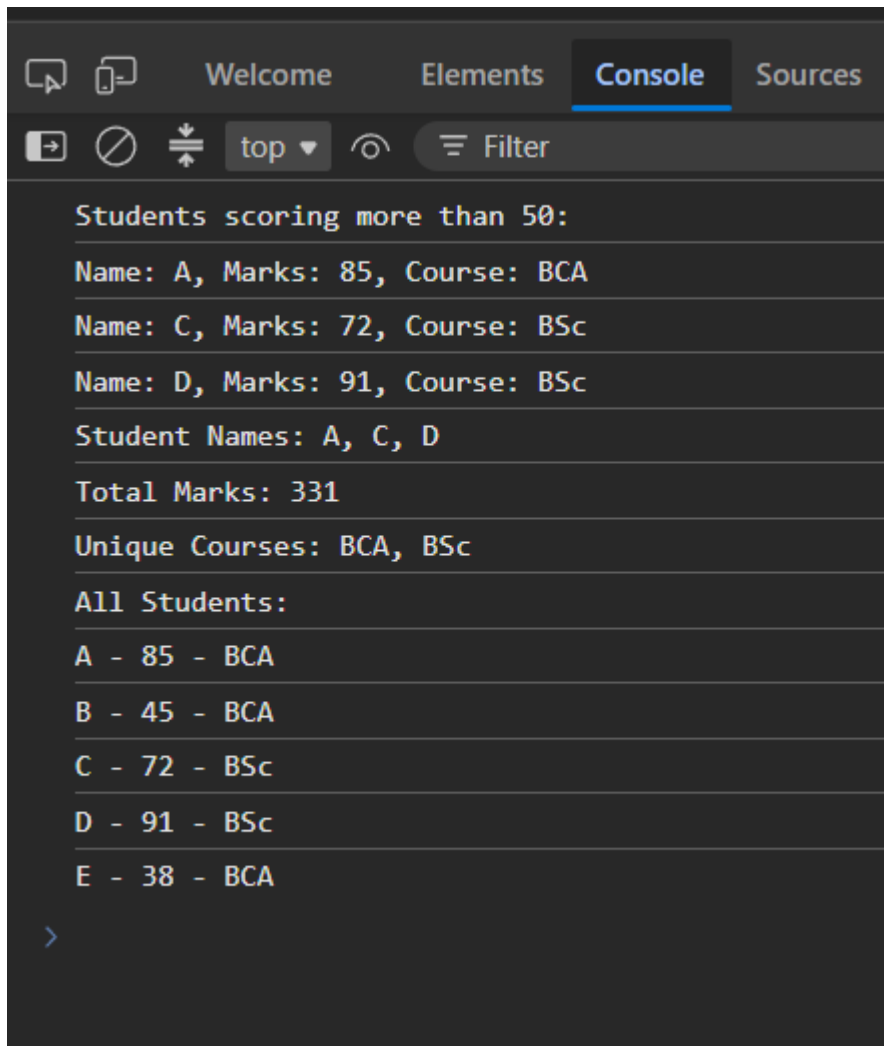
// 6. Unique courses using Set
const uniqueCourses = new Set
(
  students.map(student => student.course)
);

console.log(`Student Names: ${studentNames.join(", ")}`);
console.log(`Total Marks: ${totalMarks}`);
console.log(`Unique Courses: ${[...uniqueCourses].join(", ")}`);

// 7. Iterate through all students using for-of
console.log("All Students:");

for (const student of students)
{
  console.log
  (
    `${student.name} - ${student.marks} - ${student.course}`
  );
}
</script>
</body>
</html>
```

Output:



```
Students scoring more than 50:  
Name: A, Marks: 85, Course: BCA  
Name: C, Marks: 72, Course: BSc  
Name: D, Marks: 91, Course: BSc  
Student Names: A, C, D  
Total Marks: 331  
Unique Courses: BCA, BSc  
All Students:  
A - 85 - BCA  
B - 45 - BCA  
C - 72 - BSc  
D - 91 - BSc  
E - 38 - BCA
```